Asignatura: Diseño de Software (IEI-050) · Universidad Santo Tomás
Docente: Giovanni Cáceres R.
Líder / **Gestor de proyecto:** Nicolás Cano Leal
Equipo: Tais Montecinos, Alex Díaz
Responsable UML: Alex Díaz · **Responsable Python:** Nicolás Cano Leal
Responsable de pruebas y enlace Trello: Tais Montecinos (enlace Trello: por completar)
Fecha: 12 de noviembre de 2026 · **Versión del producto:** 0.1.0

1. Alcance

Un centro de atención necesita agendar citas para sus profesionales. Este informe documenta la iteración 1 (análisis y diseño) y la iteración 2 (producto mínimo): los cuatro diagramas UML exigidos, su traducción a código Python, la arquitectura, las pruebas con su evidencia y el estado de la tarjeta de cambio. El docente aceptó Python en lugar de Java para esta actividad.

El contrato que mantiene coherentes los diagramas, el código y las pruebas está en `docs/caso-agenda.md`; este informe lo resume y lo justifica.

2. Análisis de requisitos

Req.	Requisito	Realización
R1	Registrar profesionales, pacientes y franjas horarias de atención	`Agenda.registrar_profesional`, `registrar_paciente`, `registrar_franja`
R2	Un paciente solicita una cita con profesional, fecha y hora	mensajes 1 y 1.1 (`solicitar_cita`, `agendar_cita`)
R3	El sistema verifica que el bloque esté disponible	`Profesional.esta_disponible` (mensaje 1.1.2)
R4	Si lo está, registra la cita; si no, informa la indisponibilidad	camino `[disponible]` (1.1.3 a 1.1.5) o error `BloqueNoDisponible`

Condiciones de negocio: un profesional no puede tener dos citas activas en un mismo horario, y cada cita pertenece a un paciente y a un profesional. El enunciado no fija la duración de un bloque; se adoptó el supuesto explícito de **30 minutos**, alineados al inicio de la franja, de modo que «el bloque esté disponible» tenga un significado verificable (`DURACION_BLOQUE = 30`). `CANCELADA` figura como estado únicamente porque la regla habla de citas *activas*; no se implementa el caso de uso de cancelar, pues no está en R1 a R4.

3. Modelo de ciclo de vida y marco de gestión

Se eligió el modelo de ciclo de vida **iterativo-incremental**: el desarrollo se estructura en ciclos repetidos (iteraciones) que entregan funcionalidad parcial (incrementos). Es adecuado porque el enunciado anticipa un cambio posterior (la tarjeta de cambio llega en la última iteración), y un modelo en cascada obligaría a congelar el diseño antes de conocerlo.

- **Iteración 1:** análisis y diseño (R1 a R4 y los cuatro diagramas).
- **Iteración 2:** producto mínimo, con dos comportamientos: agendar una cita válida y bloquear dos citas del mismo profesional a la misma hora.

- **Última iteración:** incorporación de la tarjeta de cambio (sección 8).

Scrum no es un modelo de ciclo de vida sino un **marco de gestión**: es la forma de ejecutar el modelo iterativo-incremental mediante roles, eventos y artefactos. Se usa para organizar las iteraciones (tablero y tarjetas en Trello). Las técnicas puntuales (por ejemplo, el prototipado de interfaz para validar con el cliente) son herramientas dentro de ese proceso, no el modelo.

4. Diagramas

Los cuatro diagramas se dibujan en negro y reutilizan los mismos nombres de objetos y mensajes, de modo que se pueden leer en paralelo. Su fuente es `docs/generar_diagramas.py`. Cada figura ocupa una página completa para que la letra sea legible.

4.1 Diagrama de actividades

Describe el proceso principal con tres carriles: el **Paciente** ingresa profesional, fecha y hora; **PantallaAgenda** envía la solicitud y muestra el resultado; **Agenda** verifica el bloque (franja y cita activa). La decisión tiene guardas complementarias: `[bloque disponible]` conduce a crear y registrar la cita y a mostrar «Cita registrada»; `[bloque ocupado o fuera de franja]` conduce a mostrar «Bloque no disponible». Cada camino termina en su propio nodo final.

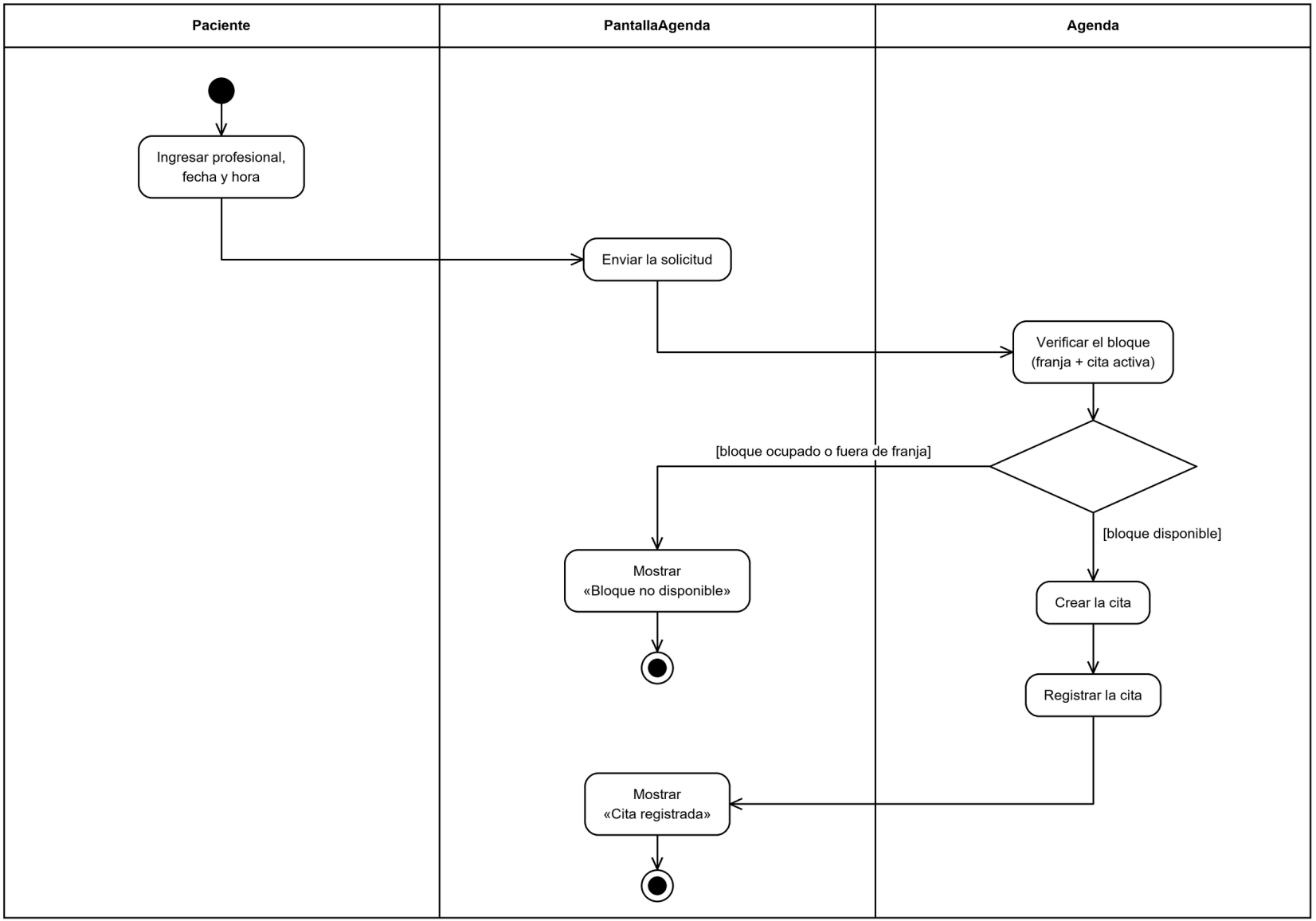


Figura 1. Diagrama de actividades: solicitud de cita con decisión de aceptación o rechazo.

4.2 Diagrama de secuencia

Muestra las instancias (`:PantallaAgenda`, `agenda:Agenda`, `:RepositorioAgenda`, `rojas:Profesional`, `c:Cita`), los mensajes en orden, las validaciones y la respuesta, para el escenario A (Juan pide a la Dra. Rojas el martes 17/11 a las 10:00). El fragmento **alt** separa los dos caminos. En `[disponible]`, `agenda` crea `c:Cita` (su cabecera aparece a la altura del mensaje «create»), la agrega a `rojas` y la guarda; en `[else]` regresa `BloqueNoDisponible` y **no existe `c:Cita`**: la cita rechazada nunca se crea.

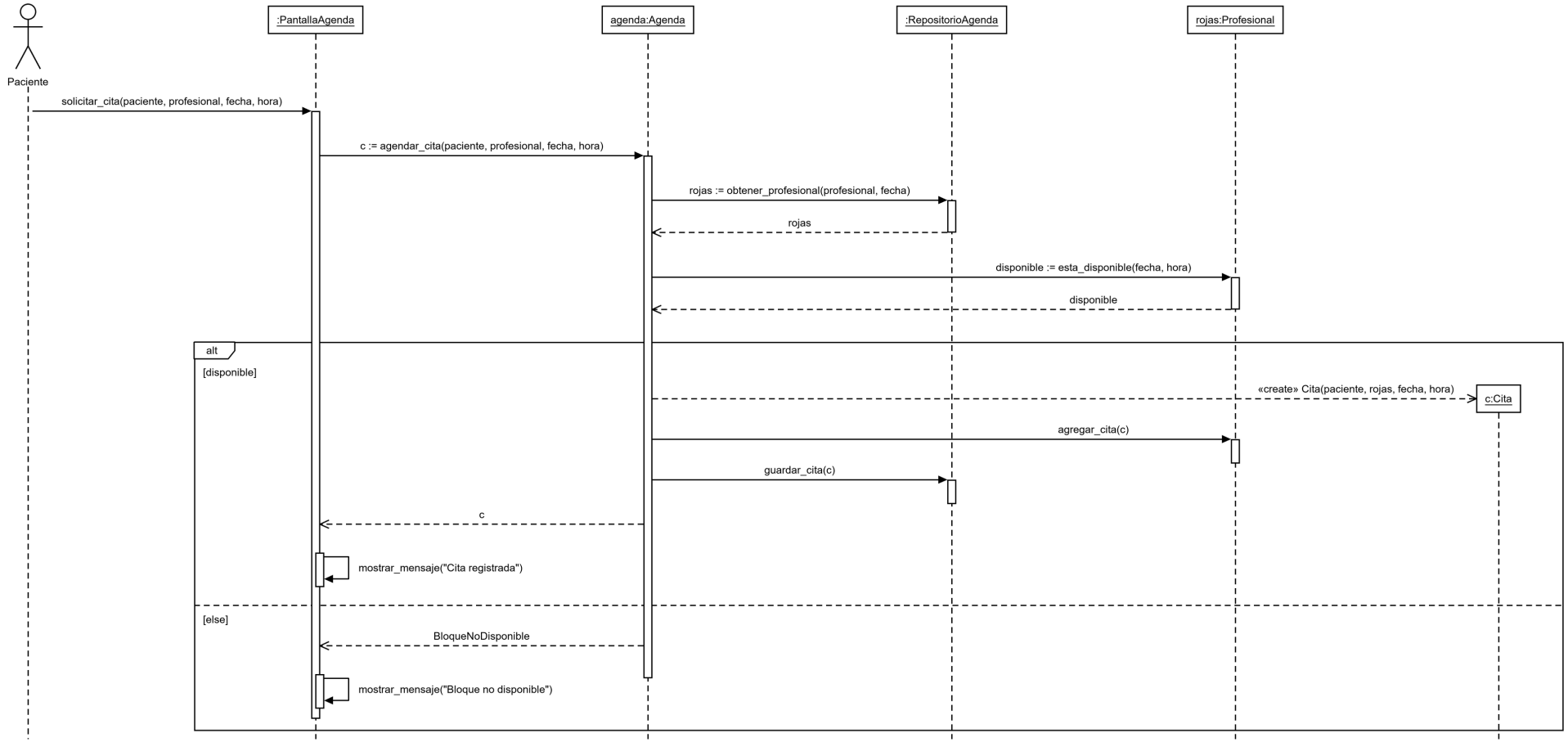


Figura 2. Diagrama de secuencia (escenario A) con fragmento alt [disponible] / [else].

4.3 Diagrama de comunicación

Contiene los mismos objetos y los mismos mensajes que la secuencia, con numeración decimal anidada: el número indica quién llamó a quién y en qué orden (1, 1.1, 1.1.1 a 1.1.5, 1.2 y 1.3). Lo que la secuencia expresa con la posición vertical, aquí lo expresa el número; lo que expresa con el fragmento alt, aquí lo expresan las guardas entre corchetes.

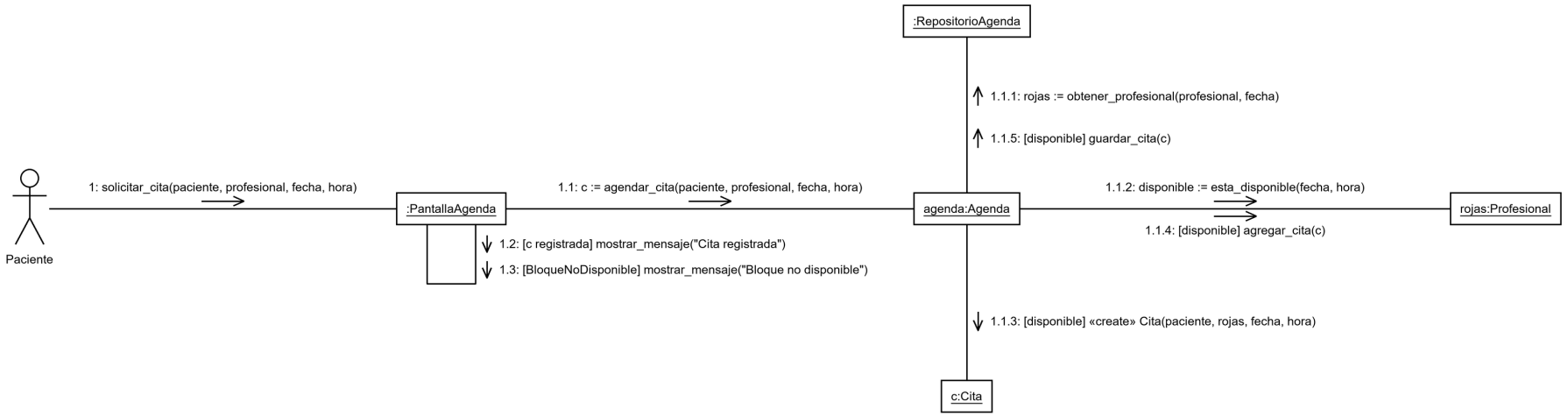


Figura 3. Diagrama de comunicación con los mensajes numerados de la secuencia.

4.4 Diagrama de clases

Incluye atributos con visibilidad y tipo, operaciones con firma completa, multiplicidades en ambos extremos y las responsabilidades de cada clase:

- **Profesional ◆ FranjaHoraria (1 a 0..*)**: composición; la franja no existe sin su profesional (en la base de datos, `ON DELETE CASCADE`).
- **Profesional — Cita (1 a 0..*)** y **Cita → Paciente (0..* a 1)**: asociaciones; cada cita pertenece a un paciente y a un profesional (condición de negocio b).
- **Cita → EstadoCita**: tipo enumerado (`ACTIVA`, `CANCELADA`).
- **Agenda ⇢ RepositorioAgenda** (dependencia) y **RepositorioPostgres, RepositorioMemoria ⇢▷ RepositorioAgenda** (realización): el caso de uso depende de la interfaz, no de la base de datos.

Responsabilidades: `Profesional` conoce sus franjas y citas y decide la disponibilidad (la regla vive en la entidad); `FranjaHoraria` sabe si contiene un bloque; `Cita` conoce su estado; `Agenda` orquesta el caso de uso; el repositorio persiste.

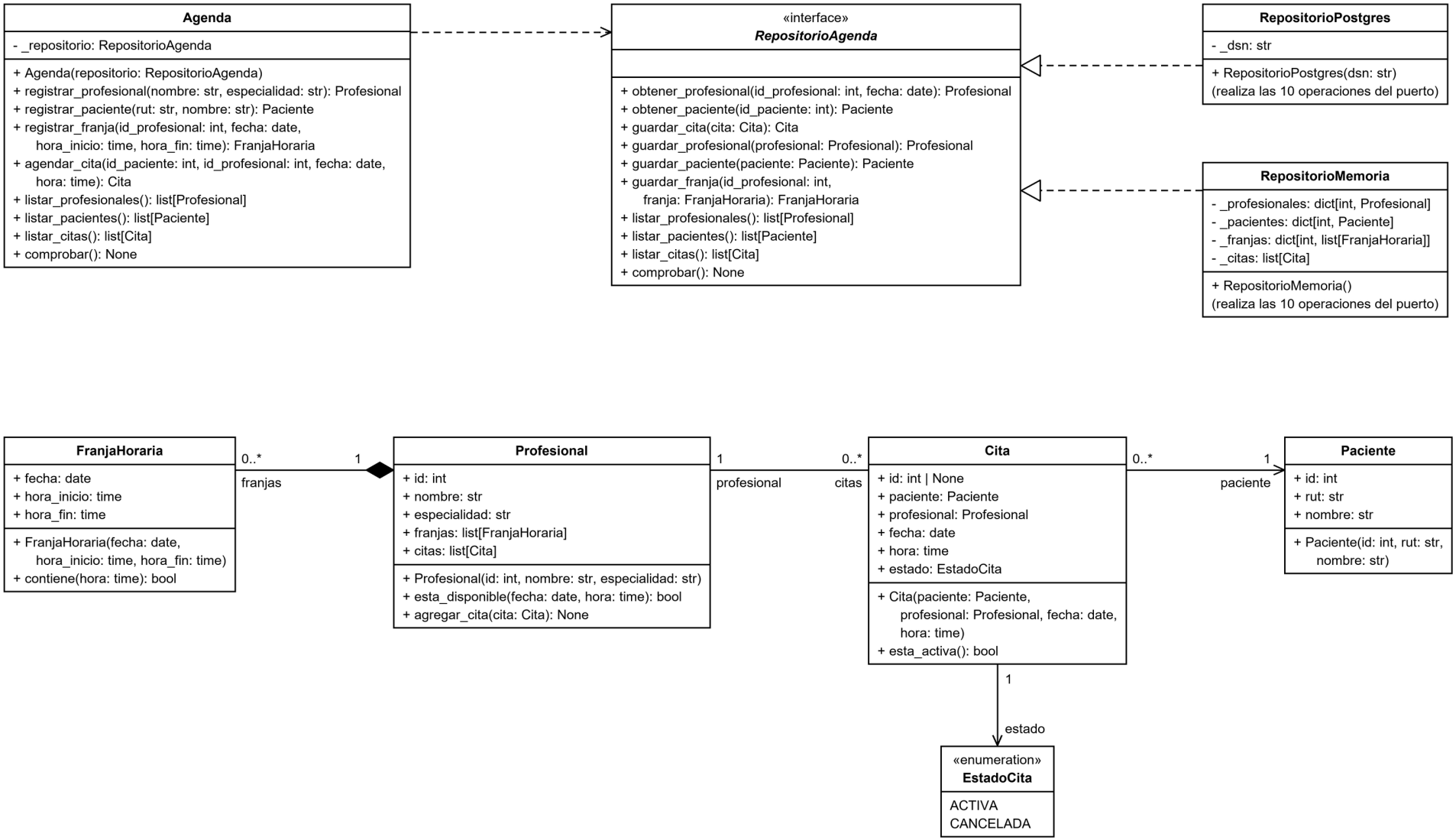


Figura 4. Diagrama de clases: dominio, caso de uso, puerto del repositorio y sus implementaciones.

5. Del diseño al código

Elemento del diseño	Código
`Profesional`, `FranjaHoraria`, `Paciente`, `Cita`, `EstadoCita`	`agenda_citas/domain/entidades.py`
Excepciones `BloqueNoDisponible`, `ProfesionalNoEncontrado`, `PacienteNoEncontrado`, `DatosInvalidos`	`agenda_citas/domain/excepciones.py`
«interface» `RepositorioAgenda`	`agenda_citas/domain/puertos.py` (`Protocol`)
`Agenda`	`agenda_citas/application/agenda.py`
`RepositorioPostgres`, `RepositorioMemoria`	`agenda_citas/adapters/outbound/`
Mensajes 1.1.1 a 1.1.5	`Agenda.agendar_cita`, en ese orden, comentadas una a una
`:PantallaAgenda`	`frontend/` (NiceGUI), vía la API de `adapters/inbound/api.py`

La regla central tiene **tres barreras**, de la más cercana al usuario a la más profunda: `Agenda.agendar_cita` consulta `esta_disponible`; `Profesional.agregar_cita` vuelve a verificar y lanza `BloqueNoDisponible`; y el índice único parcial `uq_cita_activa` de PostgreSQL rechaza una segunda cita activa si dos solicitudes concurrentes pasaron las barreras anteriores (ADR 0002). La API traduce el error de dominio a HTTP 409 con el contrato `{"error": {"codigo", "mensaje", "detalle"}}`.

6. Arquitectura y stack

La arquitectura es **hexagonal**: el dominio y los casos de uso no importan nada externo; los adaptadores de entrada (FastAPI) y de salida (PostgreSQL, memoria) dependen de ellos, y `main.py` compone las piezas. La interfaz NiceGUI consume la API por HTTP y nunca importa el dominio. El salto HTTP entre interfaz y API es un detalle de despliegue, por eso no aparece en la secuencia; sí aparece en el diagrama de clases el puerto del repositorio y su implementación.

Capa	Tecnología	Motivo
Lenguaje	Python 3.13	Aceptado por el docente; tipado y verificación con ruff
API	FastAPI	Adaptador de entrada con contrato de errores único, `/livez` y `/healthz`
Interfaz	NiceGUI	Interfaz en Python que consume la API por HTTP con *timeout*
Persistencia	PostgreSQL 17, psycopg 3	SQL explícito, migraciones versionadas e índice único parcial
Despliegue	Docker y Compose	Imagen multi-stage no-root; `db`, `api` y `ui`
Calidad	pytest, ruff, GitHub Actions	CI en tres jobs: lint, pruebas con PostgreSQL, build

Las decisiones se registran en `docs/adr/0001-python-nicegui-postgres.md` y `docs/adr/0002-indice-unico-parcial.md`.

7. Pruebas y evidencia

Las tres pruebas mínimas del enunciado, con su resultado esperado y el obtenido en la corrida registrada en `evidencias/`:

Prueba	Nombre	Resultado esperado	Resultado obtenido
P1 · escenario exitoso	`test_prueba_1_escenario_exitoso`	Juan pide a Rojas el 17/11 a las 10:00: cita registrada, estado `ACTIVA`	Aprobada
P2 · regla de negocio rechazada	`test_prueba_2_regla_doble_cita_rechazada`	María pide el mismo bloque: `BloqueNoDisponible` y ninguna cita nueva	Aprobada
P3 · caso límite	`test_prueba_3_limite_fin_de_franja`	13:00 rechazada (fuera de franja) y 12:30 aceptada (último bloque)	Aprobada

Pruebas complementarias de límites y validaciones, todas aprobadas:

Nombre	Resultado esperado
`test_hora_no_alineada_rechazada`	10:15 rechazada: no alineada a bloques de 30 minutos
`test_otro_profesional_a_la_misma_hora_aceptado`	La regla es por profesional: otro profesional a las 10:00 acepta
`test_cita_cancelada_no_bloquea`	Una cita `CANCELADA` no ocupa el bloque
`test_entidad_vuelve_a_verificar_al_agregar_cita`	`Profesional.agregar_cita` lanza `BloqueNoDisponible` por sí sola
`test_segunda_barrera_del_repositorio_ante_concurrencia`	Dos solicitudes simultáneas: la segunda falla al guardar
`test_franja_con_inicio_mayor_o_igual_al_fin_es_invalida`	`DatosInvalidos` si `hora_inicio >= hora_fin`
`test_paciente_inexistente`	`PacienteNoEncontrado` y ninguna cita creada
`test_indice_parcial_rechaza_dos_citas_activas_insertadas_directo`	PostgreSQL rechaza la segunda activa y admite una `CANCELADA`
`test_solicitudes_concurrentes_solo_una_gana`	Con dos hilos simultáneos, una cita se registra y la otra se rechaza

Evidencia JUnit. `pytest --junitxml=evidencias/junit-pruebas.xml` genera un XML con formato JUnit (Python no tiene JUnit propiamente tal; pytest produce el mismo formato) y la consola de la corrida queda en `evidencias/salida-pytest.txt`. La corrida completa ejecuta **49 pruebas, todas aprobadas** (24 unitarias, 12 de la API, 5 del cliente de la interfaz, 1 de versión y 7 de integración contra PostgreSQL 17 real), con una cobertura del **97 %** (umbral exigido: 85 %). La interfaz se verificó con un recorrido en navegador de los

escenarios A y B: `evidencias/ui-cita-registrada.png` y `evidencias/ui-bloque-no-disponible.png`.

8. Tarjeta de cambio

Pendiente. La tarjeta de cambio llega por separado en la última iteración. Se documentará aquí con su análisis de impacto sobre los cuatro diagramas, el código y las pruebas. No se anticipan requisitos más allá de R1 a R4 (sin cancelación de citas en la interfaz, autenticación ni notificaciones).

9. Organización del equipo

Rol	Responsable
Líder / Gestor de proyecto	Nicolás Cano Leal
Responsable UML	Alex Díaz
Responsable Python	Nicolás Cano Leal
Responsable de pruebas y enlace Trello	Tais Montecinos (enlace Trello: por completar)